

Vim

David Darras

08/09/2024

1 Les bases

- **Installation** : Pour installer Vim sur Ubuntu, tapez `sudo apt-get install vim`. Pour l'installer sur Windows, vous pouvez utiliser le lien suivant : <https://neovim.io/>.
- **Vim (Vi IMproved)** est un éditeur de texte puissant et hautement configurable, principalement utilisé en ligne de commande. Il est réputé pour sa rapidité et ses raccourcis clavier efficaces. Vim est une version améliorée de l'éditeur **Vi**, conçu en 1976 pour les systèmes Unix, permettant une modification rapide des fichiers sans utiliser la souris. Ce qui rend Vim unique, ce sont ses deux concepts clés : l'édition modale et les opérateurs.
- **Édition modale** : Vim utilise différents modes pour différentes tâches.
 - **Mode Normal** : Permet de taper des commandes pour naviguer et manipuler le texte. C'est le mode par défaut lorsqu'on lance Vim.
 - **Mode Insert** : Permet d'écrire du texte et est indiqué par `-- INSERT --` en bas de l'éditeur. Pour passer en mode Insert, appuyez sur `i`. Pour revenir au mode Normal, tapez `Esc` (Échap).
 - **Mode Command** : Permet de taper des commandes Ex, chercher des motifs, etc. Pour entrer dans ce mode, tapez `:`. Après avoir fini de taper la commande, appuyez sur `Enter` pour confirmer ou `Esc` pour annuler.
 - **Mode Visual** : Similaire au mode Normal - appuyez sur `v`, `V` ou `Ctrl-V`. Il permet de sélectionner des blocs de texte, mots, lignes, etc. Il est indiqué par `-- VISUAL --`.
 - **Mode Insert Normal** : Appuyez sur `Ctrl-o` en mode Insert pour exécuter une commande en mode Normal et revenir en mode Insert. Il est indiqué par `-- (insert) --`.
- Les **opérateurs** permettent de modifier, supprimer, insérer ou formater le texte.

Exemple : Pour supprimer tous les caractères situés après le curseur sur une ligne, tapez `d$`, où `d` signifie `delete` et `$` indique la fin de la ligne.
- Les différents types de commandes sont :
 - Les **commandes Ex** de la forme `<cmd>`, comme `:help`, `:wq`, etc. Pour les consulter, tapez `:help ex-cmd-index`.
 - Les **commandes Editing** sont utilisées en mode Normal, comme `d4w` pour supprimer 4 mots après le curseur.
 - Les **commandes Mapped** permettent d'associer une touche à une commande qui sera répertoriée dans le fichier `.vimrc`. Cela permet de créer ses propres raccourcis.
- **Ouvrir un fichier** : Equivalent à `Ctrl-o` sur les éditeurs de texte classique.

```
1  " Options:
2  " +NUM          : indique le numéro de ligne où placer le curseur - utile quand on débogue
3  " +/<pattern>    : va sur la ligne avec la première occurrence du pattern
4  " -x            : permet de chiffrer le fichier avec un mot de passe
5  " <filename>    : Nom du fichier à ouvrir
6  $ vim +NUM +/<pattern> +cmd -x <filename>
7
8  " ou bien
9  $ vim
10 :e <filename>
11
12 " Exemple
```

```

13 $ vim +13 main.c " Ouvre le fichier main.c et va à la ligne 13
14
15 " Ouvrir un fichier avec le curseur posé sur le nom du fichier
16 gf

```

- **Sauvegarder un fichier et quitter Vim** : Equivalent à **Ctrl-s** sur les éditeurs de texte classique.

```

1 :wq " Sauvegarde et quitte (si des modifications ont eu lieu)
2     " w - write, q - quit
3 :q! " Quitte sans sauvegarder
4 :x  " Sauvegarde et quitte seulement si des modifications ont été apportées
5 ZZ  " Équivalent à :x
6 :qa " Ferme tous les fichiers dans une session Vim, a - all

```

2 Navigation

Equivalent à se déplacer avec la souris le scroll et les fleches.

Commande	Description	Moyen mnémotechnique
h (ou ←)	Déplacement vers la gauche	h est à l'extrémité gauche
j (ou ↓)	Déplacement vers le bas	j ressemble à une demi-fleche vers le bas
k (ou ↑)	Déplacement vers le haut	logique
l (ou →)	Déplacement vers la droite	l est à l'extrémité droite
w	Aller au début du mot suivant	w ord
W	Aller au début du mot suivant en ignorant les délimiteurs	W ord
e	Aller à la fin du mot actuel	e nd
E	Aller à la fin du mot actuel en ignorant les délimiteurs	E nd
b	Aller au mot précédent	b ack
B	Aller au mot précédent en ignorant les délimiteurs	B ack
:NUM	Aller à la ligne NUM. Par exemple, :28 va à la ligne 28	NUM éro de ligne
gg	Aller en haut du fichier	g pour g o up
G	Aller en bas du fichier	G pour G o down
O	Déplacer le curseur au début de la ligne actuelle	Ligne 0 (début)
\$	Déplacer le curseur à la fin de la ligne actuelle	
H	Déplacer le curseur à la première ligne dans la fenêtre actuelle	H igh
L	Déplacer le curseur à la dernière ligne dans la fenêtre actuelle	L ow
M	Déplacer le curseur au milieu de la fenêtre actuelle	M iddle
Ctrl-d	Descendre de moitié de page	d own
Ctrl-u	Monter de moitié de page	u p pour
Ctrl-f	Descendre d'une page entière	f orward
Ctrl-b	Monter d'une page entière	b ackward
{	Aller au début du paragraphe actuel	
}	Aller à la fin du paragraphe actuel	
%	Aller à la paire correspondante de (), [],	
50%	Aller à la ligne correspondant à 50% du fichier	

- Un **mot** est une chaîne de caractères délimitée par des espaces.
- Les **délimiteurs** sont des caractères tels que les points, les parenthèses, les tirets, les virgules, les crochets, etc.
- On peut ajouter un nombre en préfixe aux commandes pour indiquer une quantité. Par exemple, 6j descend de 6 lignes, 3B revient de 3 mots en arrière, etc.
- **Il n'est pas recommandé de se déplacer en mode Insert.** Cependant, vous pouvez utiliser les flèches ainsi que **Shift- →** et **Shift- ←** pour naviguer de mot en mot.

3 Recherche textuelle

Equivalent à un `Ctrl-F` sur les autres éditeurs de texte.

Commande	Description
<code>/<pattern></code>	Rechercher un motif. Enter pour valider et Esc pour annuler
<code>?<pattern></code>	Rechercher un motif dans le sens ascendant
<code>n</code>	Aller sur la prochaine occurrence. (n ext)
<code>N</code>	Aller sur l'occurrence précédente.
<code>ggn</code>	Aller au premier match de la recherche en cours
<code>GN</code>	Aller au dernier match de la recherche en cours
<code>*</code>	Rechercher la prochaine occurrence du mot sous le curseur
<code>#</code>	Rechercher la précédente occurrence du mot sous le curseur
<code>g*</code>	Rechercher la prochaine occurrence du mot sous le curseur en utilisant une correspondance partielle
<code>g#</code>	Rechercher la précédente occurrence du mot sous le curseur en utilisant une correspondance partielle
<code>5/<pattern></code>	Sauter à la 5e occurrence du motif
<code>6*</code>	Sauter à la 6e occurrence du mot sous le curseur
<code>/ctrl-r ctrl-w</code>	Copier le mot sous le curseur comme pattern (r ead, w rite)
<code>/ ↑ ↓</code>	Naviguer dans l'historique de recherche
<code>ctrl-o</code>	Revenir à la dernière position
<code>ctrl-i</code>	Annuler le retour en arrière
<code>/</code>	Répéter la dernière recherche
<code>:s</code>	Répéter la dernière recherche et substitution
<code>:g</code>	Répéter la dernière recherche avec la commande g

Remarque : On peut aussi mettre un chiffre pour indiquer l'occurrence. `5/hello` va à la 5e occurrence de **hello**. `7*` va la 7e occurrence du mot qui est en-dessous du curseur.

4 Gestion de fichiers (netrw)

Commande	Description	Moyen mnémotechnique
<code>:Ex</code>	Ouvrir le répertoire courant	E xplore
<code>:Ex <dir></code>	Ouvrir un répertoire spécifique	E xplore d irectory
<code>:Sex</code>	Ouverture en mode horizontal	S plit
<code>:Vex</code>	Ouverture en mode vertical	V ertical e xplore
<code>:Tex</code>	Ouverture dans un nouveau tab	T ab e xplore
<code>:Lexplore</code>	Ouverture verticale sur la gauche	L eft e xplore
<code>:40vs +Ex</code>	Ouverture sur la gauche avec une largeur de 40 colonnes	
<code>\$ vim <dir></code>	Lancer netrw / Ouvrir un répertoire	
<code>Enter</code>	Ouvrir le fichier / répertoire sous le curseur	
<code>D</code>	Supprimer le fichier / répertoire	D elete
<code>R</code>	Renommer le fichier / répertoire	R ename
<code>X</code>	Exécuter le fichier	e Xecute
<code>%</code>	Créer un nouveau fichier	
<code>ctrl-w w</code>	Changer de tabview	w indow

5 Undo et Redo

Equivalent au `ctrl-z` et `ctrl-y` sur les éditeurs classiques.

Commande	Description	Moyen mnémotechnique
u	Annuler la dernière action (Undo)	U ndo
ctrl-r	Refaire la dernière action annulée (Redo)	R edo
5u	Faire 5 fois undo	Le chiffre indique la quantité d'undos
:earlier 2d	Annuler les changements des deux derniers jours	E arlier - revenir dans le passé
:ea 3h	Annuler les changements des trois dernières heures	E arly - revenir dans le passé, 3h
:ea 1m	Annuler les changements de la dernière minute	Même idée, mais pour une minute
:later 5m	Refaire tous les changements des 5 dernières minutes	L ater - avancer dans le temps
:lat 15s	Refaire tous les changements des 15 dernières secondes	L ater - avancer, ici 15 secondes
:earlier 3f	Annuler les trois derniers états de fichier	F ichiers - revenir aux états antérieurs
g+	Naviguer dans les branches d'undo (avant)	G o forward dans l'historique
g-	Naviguer dans les branches d'undo (après)	G o backward dans l'historique
:undo branches	Voir les branches d'undo disponibles	U ndo - gestion des branches
Persistent Undo	Permet de faire des undo même après fermeture de la session	Undo persistent au-delà de la session

6 Apprendre à parler Vim

- Parler Vim : (<quantité>) <verbe> <modificateur> <nom>
- Appuyez sur o pour aller à la ligne suivante et insérer du texte.
- La commande . en mode normal permet de reproduire la dernière commande.

Commande	Description
Verbes faibles	
x	Supprime le caractère sous le curseur vers la droite
X	Supprime le caractère sous le curseur vers la gauche
r	Remplace le caractère sous le curseur par un autre
s	Supprime le caractère sous le curseur et entre en mode insertion
Verbes forts	
y	Copier (y ank)
c	Changer (c hange)
d	Supprimer (d ele t e)
v	Sélectionner visuellement
Modificateurs	
i	à l'intérieur (i nn e r)
a	autour de (a roun d)
NUM	Nombre comme 3, 7, 20
t	Cherche jusqu'à l'élément trouvé (s'arrête avant)
f	Cherche l'élément et atterrit dessus
/	Recherche une chaîne de caractères ou une expression régulière
Noms (objets)	
w, W	Début du mot suivant
b, B	Début du mot précédent
e, E	Fin du mot actuel
s	Phrase
p	Paragraphe
t	Tag (en contexte HTML/XML)
b	Bloc (en contexte de programmation)
h, j, k, l	Gauche, bas, haut, droite
\$	Fin de la ligne
O	Début de la ligne
Combinaisons	
aw	Un mot complet
as	Une phrase complète
ap	Un paragraphe complet
iw	Intérieur d'un mot
is	Intérieur d'une phrase
ip	Intérieur d'un paragraphe

7 Les plages

- Par défaut, l'opération s'applique à la ligne courante.

- `:s/bad/good/g` - Remplace tous les mots "bad" par "good" dans la ligne courante.
- `:6,11s/bad/good/g` - Fait le même changement, mais entre les lignes 6 et 11 incluses.
- `:%s/bad/good/g` - Fait le même changement dans tout le fichier.

`s` signifie "substitute" (remplacer).

- `:28s/bad/good/g` - Remplace "bad" par "good" à la ligne 28.
- `:1s/bad/good/g` - Remplace "bad" par "good" à la première ligne.
- `:$s/bad/good/g` - Remplace "bad" par "good" à la dernière ligne.
- `:%s/bad/good/g` - Remplace "bad" par "good" dans tout le fichier (équivalent à `1,$`).
- `:6,28s/bad/good/g` - Remplace "bad" par "good" entre les lignes 6 et 28 incluses.
- `:11,$s/bad/good/g` - Remplace "bad" par "good" entre la ligne 11 et la fin du fichier.
- `:.,$s/bad/good/g` - Remplace "bad" par "good" de la ligne courante jusqu'à la fin du fichier.
- `:. \+1,$s/bad/good/g` - Remplace "bad" par "good" à partir de la ligne après celle courante jusqu'à la fin du fichier.
- `:. \+4s/bad/good/g` - Remplace "bad" par "good" de la ligne courante jusqu'à 4 lignes plus bas, inclusivement.
- `:?a?,/b/s/bad/good` - Remplace "bad" par "good" entre les motifs "a" et "b", inclusivement.

- **Chercher et remplacer :**

`:[range] s[substitute]/pattern/string/[flags] [count]`

Tout ce qui est entre crochets est optionnel. Les options de substitution possibles sont :

- `c` - Confirmer chaque remplacement.
- `g` - Remplacer toutes les occurrences sur la ligne.
- `i` - Ignorer la casse pour le motif.
- `I` - Ne pas ignorer la casse pour le motif.

Pour chercher et remplacer dans un fichier entier :

`:%s/old_string/new_string/g`

- `%s` signifie que la substitution sera effectuée sur toutes les lignes du fichier.
- `g` spécifie que l'action sera effectuée sur toutes les occurrences dans chaque ligne.

8 Replie ment (*fold*ing)

Commande	Description	Moyen mnémotechnique
<code>zfap</code>	Créer un repliement autour d'un paragraphe	"fold" + "ap" (around paragraph)
<code>zo</code>	Ouvrir le repliement sous le curseur	"open fold"
<code>zc</code>	Fermer le repliement sous le curseur	"close fold"
<code>za</code>	Basculer le repliement sous le curseur	"toggle fold"
<code>zd</code>	Supprimer le repliement sous le curseur	"delete fold"
<code>zj</code>	Aller au repliement suivant	"jump to next fold"
<code>zk</code>	Aller au repliement précédent	"jump to previous fold"
<code>zR</code>	Ouvrir tous les repliements dans le buffer courant	"reduce all folds"
<code>zM</code>	Fermer tous les repliements dans le buffer courant	"more folds (close all)"
<code>zE</code>	Supprimer tous les repliements dans le buffer courant	"erase all folds"
<code>:fold</code>	En mode visuel, replier les lignes sélectionnées	"fold selected"
<code>:1m2</code>	Échanger la ligne 1 avec la ligne 2	"move" ligne 1 à ligne 2
<code>20,25co10</code>	Copier les lignes de 20 à 25 sous la ligne 10	"copy" lignes

- `co` pour "copy" (copier), et `m` pour "move" (déplacer).

9 Divers

Commande	Description	Moyen mnémotechnique
30i- 3fq 0 \$ o O x r v puis e puis d :help yy P	Insérer 30 fois le caractère - Chercher la 3 ^e occurrence de q Aller au début de la ligne Aller à la fin de la ligne Insérer une ligne après Insérer une ligne avant Supprimer le caractère sous le curseur Remplacer un caractère sous le curseur Sélectionner un mot Ouvrir l'aide Vim Copier la ligne courante Coller avant le curseur	"i" pour insérer, suivi de répétition "find q", 3 fois "début de la ligne" "fin de la ligne" (comme un signe de fin) "open line below" "Open line above" "x" pour supprimer "replace" "visual select, extend, delete" "help" pour assistance "yank yank" (double pour copier toute la ligne) "Paste" avant le curseur
Ctrl-V c Esc	Sélectionner un bloc de texte avec les flèches Changer le bloc sélectionné par de nouveaux caractères Sortir du mode d'insertion	"Visual Block" "change" "Escape" pour quitter
:r file.txt :Or file.txt :r!sed -n 2,8p file.txt :r !ls	Insérer le contenu de file.txt sous le curseur Insérer file.txt avant la première ligne Insérer les lignes 2 à 8 de file.txt sous le curseur Insérer la liste des fichiers du répertoire courant	"read file" "read file at start" "read specific lines" "read output of ls"

Exemple de remplacement de bloc de caractères :

1. Appuyez sur Ctrl-V pour entrer en mode bloc visuel.
2. Sélectionnez le bloc à modifier avec les flèches.
3. Appuyez sur c pour changer et tapez la nouvelle chaîne.
4. Enfin, appuyez sur Esc pour terminer.

```
// Avant
Bonjour --> Hello
Au revoir --> Bye
Merci --> Thank you
```

```
// On sélectionne le bloc fait de "--" et on tape "="
// Après
Bonjour => Hello
Au revoir => Bye
Merci => Thank you
```

10 Configurer Vim

- Vous pouvez modifier le fichier ~/.vimrc pour spécifier vos préférences au lieu de retaper ces commandes à chaque fois

```
1 :set number           " Affiche les numéros de ligne
2 :set nonumber        " Désactive les numéros de ligne
3 :set number!         " Toggle les numéros
4 :set cursorline      " Affiche une ligne sous le curseur
5                     " Modifiable avec :highlight CursorLine etc.
6 :set cursorcolumn    " Affiche la colonne du curseur
7 :set spell           " Ajoute un correcteur orthographique avec
8 :set spelllang=en,fr,de " Ajoute d'autres langues que l'anglais
9                     " pour savoir si une option est activé :set spell?
10 :colorscheme <nom>   " Modifier le theme (les couleurs)
11 " etc.
```

- On peut créer ses propres raccourci claviers avec les mappings mais ce n'est pas abordé ici.
- Il est aussi possible d'installer des plugins (cf Vim-Vundle).